Aula 35: Componentes de Alta Ordem (HOC) e Render Props

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

Índice

- INTRODUÇÃO
- FORMULÁRIO EM REACT
- INPUTS CONTROLADOS
- O PROBLEMA DA ESCALABILIDADE
- UNCONTROLLED COMPONENTS & REFS
- VALIDAÇÃO MANUAL E FEEDBACK AO USUÁRIO
- VALIDAÇÃO COM BIBLIOTECAS
- REACT HOOK FORM (RHF)
- VALIDAÇÃO COM ZOD
- FORMULÁRIOS COMPLEXOS
- EXERCÍCIOS

Introdução

- Em aplicações React de grande escala, frequentemente precisamos compartilhar a mesma funcionalidade entre vários componentes independentes
- O desafio é como evitar o "Copy-Paste" de código, mantendo a manutenção simples e o código limpo
- DRY - Don't Repeat Yourself

Introdução

- Cenários Comuns:
  - Autenticação e Autorização
  - Registro de logs (Logging)
  - Acesso a dados de APIs
  - Manipulação de formulários

Introdução

EXEMPLO

- Cenário: Dois componentes precisam da mesma lógica (ex: rastrear posição do mouse ou verificar se o usuário está autenticado)
- Problema: Copiar e colar código fere o princípio DRY (Don't Repeat Yourself)
- Solução: Abstrair a lógica em um único lugar

Introdução

COMPOSIÇÃO VS. HERANÇA

- React favorece composição
- Componentes recebem comportamento via props

PROBLEMA COMUM

- Mesma lógica repetida em vários componentes
- Ex.: autenticação, carregamento, mouse position, permissões

Componentes de Alta Ordem (HOC)

- Um Higher-Order Component (HOC) é uma técnica avançada no React para reutilizar a lógica de componentes
- Função que recebe um componente e retorna outro componente
- Este padrão deriva das Higher-Order Functions do JavaScript

```
const EnhancedComponent = higherOrderComponent(WrappedComponent);
```

- Onde withFeature é a função HOC que injeta novas props ou comportamentos

Componentes de Alta Ordem (HOC)

CARACTERÍSTICAS

- Não modifica o componente original
- Em vez disso, ele "envolve" o componente original em um componente de contêiner
- Injeta props adicionais
- Padrão funcional

Componentes de Alta Ordem (HOC)

EXEMPLO DE USO

```
function withLogging(WrappedComponent) {
  return class extends React.Component {
    componentDidMount() {
      console.log('Componente montado!');
    }
    render() {
      return <WrappedComponent {...this.props} />;
    }
  };
}
```

Componentes de Alta Ordem (HOC)

EXEMPLO DE USO

```
function withLogging(WrappedComponent) {
  return class extends React.Component {
    componentDidMount() {
      console.log('Componente montado!');
    }
    render() {
      return <WrappedComponent {...this.props} />;
    }
  };
}
```

Ponto Chave: Sempre passe as `props` adiante (`{...this.props}`).

Exemplo Prático

- Realizar o exercício de HOC do W3Schools
- Entrar no site:

https://www.w3schools.com/REACT/react_hoc.asp

- Ler a seção e realizar o exercício

Vantagens e Desvantagens do HOC

VANTAGENS

- Reutilização de Lógica: Compartilha funcionalidades entre componentes
- Separação de Preocupações: Separa lógica de negócio da apresentação

DESVANTAGENS

- Wrapper Hell: O aninhamento excessivo de HOCs complica a depuração
- Conflitos de Props: Risco de sobrescrever props entre HOCs
- Origem Obscura: Dificulta rastrear a origem das props

Vantagens e Desvantagens do HOC

BOAS PRÁTICAS E CUIDADOS

- Convenções:
  - Nome: withAlgo
  - Passar todas as props ({...props})
- Problemas comuns
  - Wrapper Hell (muitos HOCs aninhados)
  - Dificuldade de debug
  - Colisão de nomes de props

Render Props

- O termo Render Prop refere-se a uma técnica de compartilhar código entre componentes React usando uma prop cujo valor é uma função
- Diferente do HOC, que "envolve" o componente, as Render Props permitem que o componente pai diga ao componente filho o que renderizar
- **Conceito**: Um componente com uma render prop recebe uma função que retorna um elemento React e a chama em vez de implementar sua própria lógica de renderização.

Render Props

- Ou seja, uma Render Prop é um componente que recebe uma função como prop
- Essa função define o que será renderizado

```
<Component render={(data) => <UI data={data} />} />
```

```
<DataProvider
  render={data => (
    <h1>{data.title}</h1>
  )}
/>
```

Render Props

- Ou seja, uma Render Prop é um componente que recebe uma função como prop
- Essa função define o que será renderizado

```
<Component render={(data) => <UI data={data} />} />
```

```
<DataProvider
  render={data => (
    <h1>{data.title}</h1>
  )}
/>
```

A prop render (ou qualquer outro nome) é uma função que recebe os dados e retorna o JSX.

Render Props

EXEMPLO CONCEITUAL

- Caso típico
  - Compartilhar lógica de mouse, scroll ou dimensões da tela
- Ideia central
  - O componente controla a lógica
  - Quem usa decide como renderizar

Render Props

VANTAGENS E DESVANTAGENS

- Vantagens
  - Mais explícito que HOC
  - Menos colisão de props
- Desvantagens
  - JSX mais verboso
  - Muitas funções inline

Exemplo Prático

- Compartilhando a posição do mouse de forma dinâmica:

```
class Mouse extends React.Component {
  state = { x: 0, y: 0 };
  handleMouseMove = e => this.setState({ x: e.clientX, y: e.clientY });

  render() {
    return (
      <div onMouseMove={this.handleMouseMove}>
        {this.props.render(this.state)}
      </div>
    );
  }
}
<Mouse render={m => <h1>Posição: ({m.x}, {m.y})</h1>} />
```

HOC vs Render Props

Critério	HOC	Render Props
Injeção de Dados	Via Props (pode haver conflitos)	Via Argumentos (explícito)
Estrutura JSX	Estática (envolve o componente)	Dinâmica (função no render)
Legibilidade	Pode causar "Wrapper Hell"	Pode causar "Callback Hell"
Melhor para...	Lógica global (Auth, Logs)	Lógica de UI (Mouse, Scroll)

- **Dica**: Embora ambos sejam fundamentais, os React Hooks substituíram a maioria desses casos de uso em componentes funcionais
- Use HOC/Render Props em componentes de classe ou bibliotecas complexas

Exercício

- HOC: O Decorador de Logger
- Objetivo:
  - Criar um HOC chamado withLogger que imprime no console o nome do componente sempre que ele for renderizado.
- Use WrappedComponent.displayName ou WrappedComponent.name

Exercício

- Render Props: O Componente de Relógio
- Objetivo:
  - Criar um componente <Timer /> que gerencia um intervalo (segundos passados)
  - Ele deve aceitar uma render prop para que o aluno decida se quer exibir o tempo em um <h1>, em um input ou apenas um alerta quando chegar a 10 segundos

Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Componentes de Alta Ordem (HOC)**
  - Criar uma página "Estatísticas" (Quantas tarefas fiz na semana?)
  - Criar um HOC chamado withPremiumAccess
  - Esse HOC verifica uma variável (pode ser hardcoded const isPremium = false). Se for false, mostra "Faça upgrade para ver isso". Se true, mostra o componente.
- Funcionalidade: Criar uma proteção de rota ou lógica reutilizável

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Componentes de Alta Ordem (HOC)**
- Resultado Visual:
  - Uma rota protegida que exibe conteúdo condicionalmente baseado em uma lógica encapsulada.
- Conceito chave: Reutilização de lógica